Programación 1 — Unidad 8: Manejo de errores

Actividad 2: Excepciones en Python

Índice

– 8.2. El bloque try / except

– 8.3. Capturar varias excepciones

– 8.4. Los bloques else y finally

– 8.5. Obtener información del error y crear excepciones propias

8.2. El bloque try / except

La herramienta central es la estructura `try` / `except`. El código «riesgoso» (el que podría fallar) se pone dentro del `try`. Si ocurre una excepción, en lugar de romperse, Python salta al bloque `except`, que decide qué hacer.

```
try:
    edad = int(input("Ingresá tu edad: "))
    print(f"El año que viene tendrás {edad + 1}")
except ValueError:
    print("Eso no es un número válido.")
```

Qué hace el código:

- Dentro del `try:` va lo que podría fallar: convertir a número lo que escriba la persona.
- Si escribe un número, todo funciona y el `except` se ignora por completo.
- Si escribe, por ejemplo, «hola», `int()` falla y se lanza un `ValueError`.
- `except ValueError:` atrapa justo ese error y ejecuta su mensaje, en vez de cortar el programa.

Analogía

El `try`/`except` es como la red de seguridad de un trapecista. El artista intenta la pirueta (el `try`); si sale bien, la red no se usa. Pero si falla, la red lo atrapa (el `except`) y evita el desastre. El espectáculo continúa en lugar de terminar de golpe.

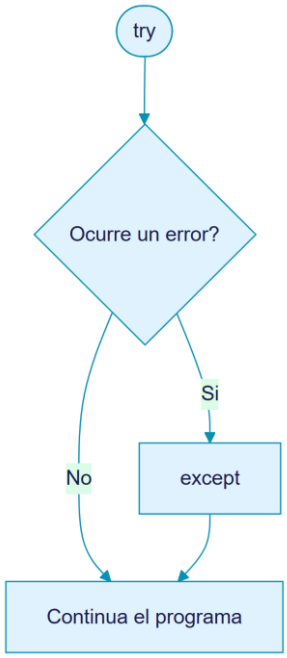


Diagrama 8.2 generado con Mermaid.

8.3. Capturar varias excepciones

Una misma porción de código puede fallar de distintas maneras. Podemos escribir varios bloques `except`, uno por cada tipo de error, para responder de forma específica a cada uno. Python ejecuta el primero que coincida.

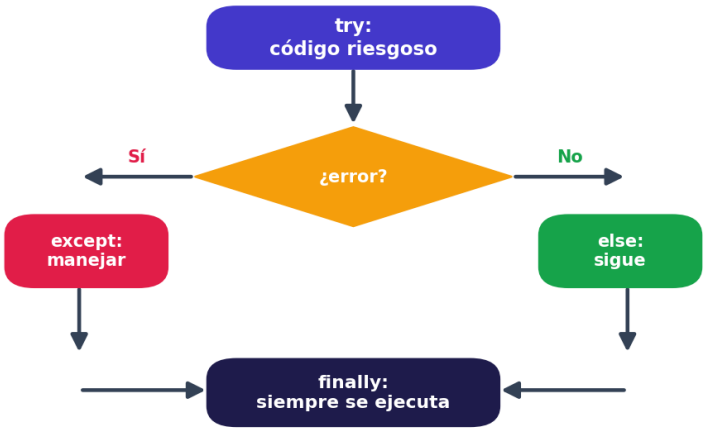
```
try:
    a = int(input("Numerador: "))
    b = int(input("Denominador: "))
    print(a / b)
except ValueError:
    print("Tenés que ingresar números enteros.")
except ZeroDivisionError:
    print("No se puede dividir por cero.")
```

Qué hace el código:

- El `try` pide dos números y los divide: hay dos cosas que pueden salir mal.
- Si la persona escribe algo que no es número, salta el `except ValueError`.
- Si el denominador es 0, la división falla y salta el `except ZeroDivisionError`.
- Cada error tiene su propio mensaje, así la persona entiende exactamente qué pasó.

8.4. Los bloques else y finally

La estructura se completa con dos bloques opcionales. El `else` se ejecuta **solo si no hubo** ninguna excepción en el `try`. El `finally` se ejecuta **siempre**, haya error o no, y se usa para tareas de limpieza (como cerrar un archivo).



Flujo completo: el try intenta; si hay error va al except, si no al else; el finally corre en todos los casos.

```
try:
    numero = int(input("Ingresá un número: "))
except ValueError:
    print("No era un número.")
else:
    print(f"El doble es {numero * 2}")
finally:
    print("Gracias por usar el programa.")
```

Qué hace el código:

- El `try` intenta convertir lo ingresado a número.
- Si falla, se ejecuta el `except` con su mensaje de error.
- Si **no** falla, se ejecuta el `else`: muestra el doble del número (esa parte solo corre cuando todo salió bien).
- El `finally` se ejecuta en todos los casos, con o sin error: por eso el agradecimiento aparece siempre.

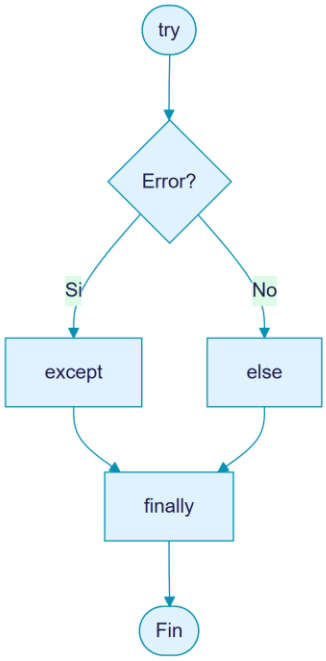


Diagrama 8.4 generado con Mermaid.

8.5. Obtener información del error y crear excepciones propias

Al capturar una excepción podemos guardarla en una variable con `as` para ver su mensaje. Además, podemos **lanzar** errores a propósito con `raise`, e incluso definir nuestras propias excepciones creando una clase que herede de `Exception`.

```
class SaldoInsuficiente(Exception):
    pass

def extraer(saldo, monto):
    if monto > saldo:
        raise SaldoInsuficiente("No hay saldo suficiente")
    return saldo - monto

try:
    extraer(1000, 5000)
except SaldoInsuficiente as error:
    print("Rechazado:", error)
```

Qué hace el código:

- `class SaldoInsuficiente(Exception):` define una excepción propia, con un nombre claro para nuestro problema.
- Dentro de `extraer`, si el monto supera el saldo, `raise SaldoInsuficiente(...)` lanza ese error a propósito.
- El `try` llama a `extraer` con un monto demasiado grande, lo que dispara la excepción.
- `except SaldoInsuficiente as error:` la atrapa y, gracias al `as error`, puede mostrar el mensaje que le pusimos.

Buenas prácticas

No uses `except:` a secas para atrapar «cualquier cosa» y seguir como si nada. Capturá los errores específicos que esperás; así no escondés errores reales y tus mensajes son útiles. Manejar errores no es disimularlos, es responder a ellos.

Encontrá el error

Este programa ni siquiera llega a ejecutarse. Mirá con atención la línea del except antes de seguir.

```
try:
    numero = int("abc")
except ValueError as e
    print("Error:", e)
```

Qué estaba mal

Falta los dos puntos (:) al final de la línea except ValueError as e. Es un SyntaxError: hasta en el manejo de errores hay que respetar la sintaxis del lenguaje.

Practicá la lectura

Sin ejecutar, escribí en orden todo lo que se va a imprimir, explicando cuándo se ejecuta el bloque else y cuándo el finally.

```
try:
    numero = int("42")
except ValueError:
    print("Error de conversion")
else:
    print("Conversion exitosa:", numero)
finally:
    print("Fin del bloque")
```

Para reflexionar y autoevaluarte

— ¿Por qué te parece mala práctica usar un except sin especificar el tipo de error?

— ¿Se te ocurre alguna situación de tus propios programas donde convendría crear una excepción propia?

Antes de seguir, verificá que podés:

— Usar try / except para manejar errores específicos.

— Explicar el rol de los bloques else y finally.

— Capturar más de un tipo de excepción en el mismo bloque.

— Crear una excepción propia cuando el caso lo amerita.